Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 7

Three Small Programs: Processes, Redirection, a Pipe

Track A: the real OS, from the outside (Linux)

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. fork + exec + wait **2**

2.1 Program structure 2

2.2 Running `ls -l` 3

2.3 Running `date` 3

2.4 Failed `exec` 3

2.5 Observations 4

3 Part 2. A Zombie, on Purpose **4**

3.1 Observations 5

4 Part 3. Redirection: `ls -l > out.txt` **5**

4.1 Observations 6

5 Part 4. A Pipe: `ls /etc | wc -l` **6**

5.1 Broken-pipe experiment 7

5.2 Observations 7

6 Conclusion **8**

1. Introduction

The purpose of this laboratory work is to study the basic mechanisms used by a Linux shell when it starts programs, redirects output, and connects processes with a pipe. The work consists of four small C programs demonstrating process creation with `fork()`, program replacement with `exec()`, process synchronization with `waitpid()`, zombie processes, file descriptor redirection, and inter-process communication through a pipe.

The experiments were performed on Ubuntu under WSL2 in the working directory `/õs-lab7`. The installed compiler was GCC 13.3.0.

```
$ gcc --version | head -1
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0

$ pwd
/home/sofia/os-lab7
```

2. Part 1. fork + exec + wait

The first program reproduces the main steps a shell performs when executing a command such as `ls -l`: it creates a child process with `fork()`, replaces the child with another program using `execlp()`, and waits for the child in the parent using `waitpid()`.

2.1. Program structure

The essential process-control fragment was:

```
pid_t pid = fork();

if (pid == 0) {
    printf("child: my PID is %d, my parent is %d\n",
           getpid(), getppid());
    fflush(stdout);

    execlp("ls", "ls", "-l", (char *)NULL);
    perror("exec");
    exit(127);
}

int status;
```

```
waitpid(pid, &status, 0);
printf("parent: child %d ended with status %d\n",
       pid, WEXITSTATUS(status));
```

2.2. Running `ls -l`

The initial execution produced:

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 41031
child: my PID is 41032, my parent is 41031
total 24
-rwxr-xr-x 1 sofia sofia 16400 Oct 19 14:10 run
-rw-r--r-- 1 sofia sofia   919 Oct 19 14:09 run.c
parent: child 41032 ended with status 0
```

The parent process had PID 41031. After `fork()`, the new child had PID 41032, and its parent PID was correctly reported as 41031. The child then became `ls -l`. The parent waited for it and received exit status 0.

2.3. Running `date`

The `execlp()` line was changed to:

```
execlp("date", "date", (char *)NULL);
```

The result was:

```
parent: my PID is 41555
child: my PID is 41556, my parent is 41555
Mon Oct 19 14:11:58 EEST 2026
parent: child 41556 ended with status 0
```

The child process was replaced by `date` while keeping its PID, and the command finished successfully with status 0.

2.4. Failed exec

The child was then asked to execute a non-existing program:

```
execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

The result was:

```
parent: my PID is 42078
```

```
child: my PID is 42079, my parent is 42078
exec: No such file or directory
parent: child 42079 ended with status 127
```

2.5. Observations

1. `perror("exec")` is printed only when `execlp()` fails. A successful `exec` replaces the current process image, so execution never returns to the next line of the old program.
2. The value printed by the child as “my parent” is the PID of the parent process. For example, child PID 41032 reported parent PID 41031.
3. The parent saw exit status 0 for `ls` and `date`. For `nosuchprog`, the child executed `exit(127)`, so the parent received status 127.

3. Part 2. A Zombie, on Purpose

A zombie is a process that has already terminated but whose parent has not yet collected its termination status with `wait()` or `waitpid()`. To create one intentionally, the parent was made to sleep for 20 seconds before calling `waitpid()`.

```
int status;
sleep(20);
waitpid(pid, &status, 0);
```

The program was compiled and started in the background:

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  40200   40193 Ss   -bash
  44268   40200 S    ./zombie
  44270   44268 Z    [ls] <defunct>
  44271   40200 R+   ps -o pid,ppid,stat,cmd
```

The child `ls` had state `Z`, meaning zombie. It had already finished, but the parent process was still sleeping.

After the 20-second delay, another process listing showed:

```
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  40200   40193 Ss   -bash
  44456   40200 R+   ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
```

3.1. Observations

1. The `ls` process had `STAT = Z`. This means that the child had terminated and become a zombie while its parent had not yet collected its exit status.

2. After 20 seconds, the `[ls] <defunct>` entry disappeared. The call `waitpid(pid, &status, 0)` collected the child status and removed the zombie entry from the process table.

4. Part 3. Redirection: `ls -l > out.txt`

Linux processes normally start with three standard file descriptors: 0 for standard input, 1 for standard output, and 2 for standard error. The program manually redirected descriptor 1 to a file.

The important part of the program was:

```
close(1);

int fd = open("out.txt",
              O_WRONLY | O_CREAT | O_TRUNC,
              0644);

fprintf(stderr, "open gave descriptor %d\n", fd);
execlp("ls", "ls", "-l", (char *)NULL);
```

After descriptor 1 was closed, `open()` reused the smallest available file descriptor, which was 1. Therefore, the new file `out.txt` became standard output for the child process.

The execution produced:

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
```

The listing itself did not appear on the screen. It was stored in `out.txt`:

```
$ head -3 out.txt
total 68
-rw-r--r-- 1 sofia sofia     0 Oct 19 14:26 out.txt
-rwxr-xr-x 1 sofia sofia 16344 Oct 19 14:26 redirect
```

4.1. Observations

1. `open()` returned descriptor 1 because `close(1)` had made descriptor 1 free, and the kernel assigns the smallest available descriptor.
2. `ls` itself was not modified. It continued to write to standard output, descriptor 1. However, descriptor 1 now referred to `out.txt`, so the listing was written to the file rather than the terminal.
3. To redirect errors in the same way as shell syntax `2> err.txt`, descriptor 2 would need to be closed before the call to `open()`.

5. Part 4. A Pipe: `ls /etc | wc -l`

A pipe connects the output of one process to the input of another. The call `pipe(p)` creates two descriptors: `p[0]` for reading and `p[1]` for writing.

The first child was configured so that its standard output became the write end of the pipe:

```
if (fork() == 0) {
    close(1);
    dup(p[1]);
    close(p[0]);
    close(p[1]);
    execlp("ls", "ls", "/etc", (char *)NULL);
    perror("exec ls");
    exit(127);
}
```

The second child was configured so that its standard input became the read end of the pipe:

```
if (fork() == 0) {
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    execlp("wc", "wc", "-l", (char *)NULL);
```

```
    perror("exec wc");
    exit(127);
}
```

The parent closed both original pipe ends and waited for both children:

```
close(p[0]);
close(p[1]);
wait(NULL);
wait(NULL);
```

The manually constructed pipeline gave:

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
183
```

The real shell pipeline produced the same value:

```
$ ls /etc | wc -l
183
```

5.1. Broken-pipe experiment

The parent was modified temporarily so that it did not close its copy of the write end:

```
close(p[0]);
/* close(p[1]); */
```

After recompilation, the program did not terminate and had to be stopped with `Ctrl+C`:

```
$ gcc -Wall -Wextra -o pipeline pipeline.c
$ ./pipeline
^C
```

The reason is that `wc` waits for end-of-file on the pipe. EOF is generated only after every write descriptor referring to the pipe has been closed. Because the parent still kept `p[1]` open, `wc` continued to wait for possible additional input.

5.2. Observations

1. The manually created pipeline and the shell pipeline produced the same number: 183.

2. After the parent write end was deliberately left open, `wc` was the child process that did not terminate.

3. `wc` was waiting for EOF. EOF could not arrive while the parent still had a write end of the pipe open.

6. Conclusion

In this laboratory work, the internal behavior of several basic shell operations was reproduced using short C programs. The `fork()` system call created child processes, `execlp()` replaced a child with another program, and `waitpid()` synchronized the parent with the child and removed zombie processes after termination.

The work also demonstrated that Linux I/O is based on file descriptors. By closing descriptor 1 and opening a file in its place, standard output was redirected without modifying the `ls` program itself. Finally, a pipe was constructed manually using two child processes, descriptor duplication, and correct closing of unused pipe ends. The final experiment showed why leaving a write end open prevents the reader from receiving EOF and causes the pipeline to wait indefinitely.